

Informe del análisis del caso realizado

Automatización QA con Python, pytest y Selenium Grid

Curso	INF-37 - Herramientas para el Desarrollo de Sistemas de Información
Docente	Federico Sánchez Diaz
Estudiante	Miguel Alejandro Fernandez Arteaga
Fecha	16/07/2026
Porcentaje del caso	5%
Sitio seleccionado	Practice Test Automation

1. Descripción del caso

El objetivo del caso consiste en desarrollar una prueba de automatización QA con Selenium, usando un lenguaje de programación seleccionado por el estudiante. Para esta entrega se eligió Python con pytest, conectado a Selenium Grid ejecutado mediante Docker.

El sitio utilizado fue Practice Test Automation, disponible en el anexo del caso. Se escogió este sitio para realizar una solución distinta al ejemplo de SauceDemo y cubrir escenarios de login valido e invalido.

2. Parte 1 - Selenium Docker en ejecución

Se verifico la ejecución de Selenium Grid por medio de los endpoints locales requeridos en el enunciado. El servicio de Grid reporto estado listo y el servicio noVNC respondió correctamente.

Grid	http://localhost:4444/status - ready: true - Selenium Grid ready.
Interfaz Grid	http://localhost:4444/ui
noVNC	http://localhost:7900 - HTTP 200 OK
Navegador remoto	Chrome 149.0 sobre Linux aarch64

Selenium Grid - Evidencia de ejecucion

Endpoint consultado	http://localhost:4444/status
Estado general	ready: true
Mensaje	Selenium Grid ready.
Nodo	availability: UP
Version Selenium	4.45.0
Navegador remoto	Chrome 149.0
Sistema del contenedor	Linux aarch64
Puerto noVNC	7900

Validacion tecnica usada para confirmar que Docker/Selenium estaba disponible antes de ejecutar py

Figura 1. Selenium Grid disponible en Docker.



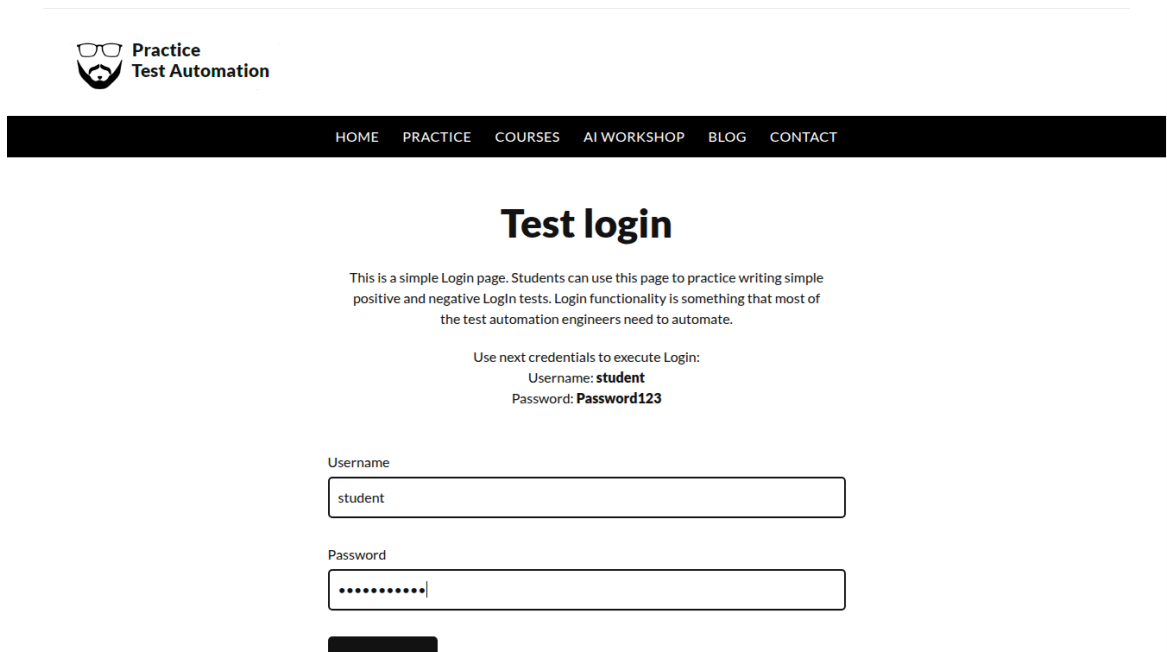
Figura 2. Interfaz noVNC disponible para visualizar el navegador remoto.

3. Parte 2 - Casos de prueba automatizados

La automatización fue implementada en Python con pytest. Las pruebas se conectan a Selenium Grid por medio de `http://localhost:4444/wd/hub` y ejecutan acciones sobre el formulario de login del sitio seleccionado.

Caso	Datos	Resultado esperado	Resultado obtenido
Login valido	student / Password123	Acceso correcto al area protegida.	Prueba aprobada. Se muestra mensaje de login exitoso y boton Log out.
Login invalido	usuario_prueba / clave_equivocada	Acceso denegado y mensaje de error visible.	Prueba aprobada. Se muestra el mensaje Your username is invalid!

4. Evidencias de ejecución



Practice Test Automation

HOME PRACTICE COURSES AI WORKSHOP BLOG CONTACT

Test login

This is a simple Login page. Students can use this page to practice writing simple positive and negative Login tests. Login functionality is something that most of the test automation engineers need to automate.

Use next credentials to execute Login:
Username: **student**
Password: **Password123**

Username
student

Password
.....

Login

Figura 3. Formulario con credenciales validas antes de enviar.

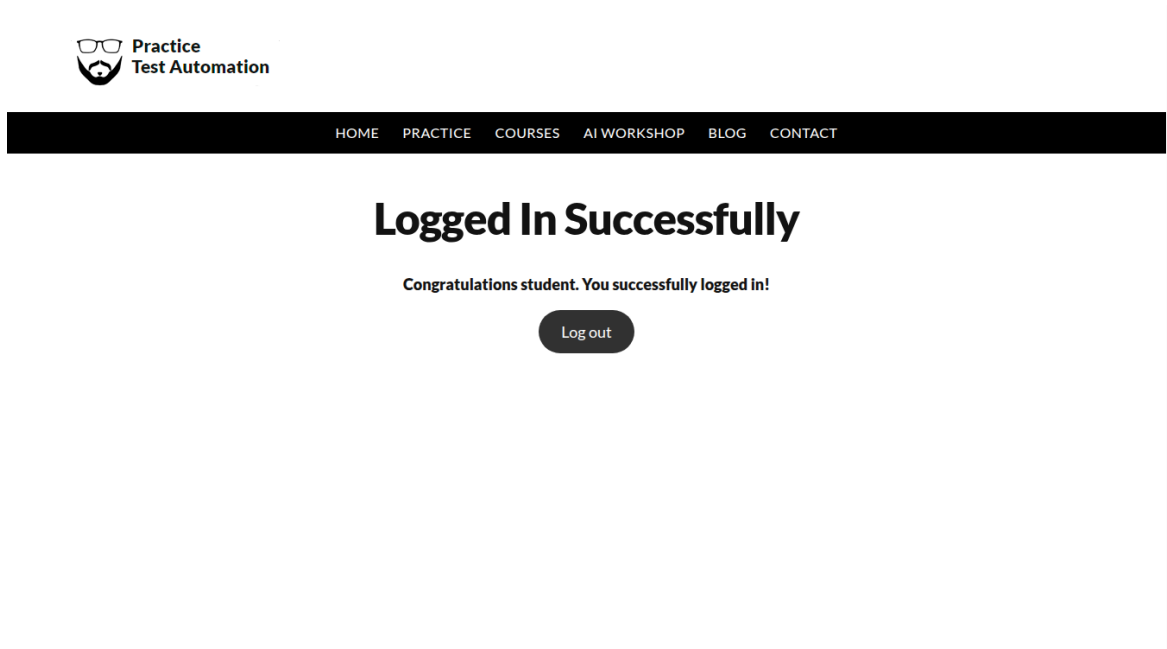


Figura 4. Login valido confirmado en la página protegida.

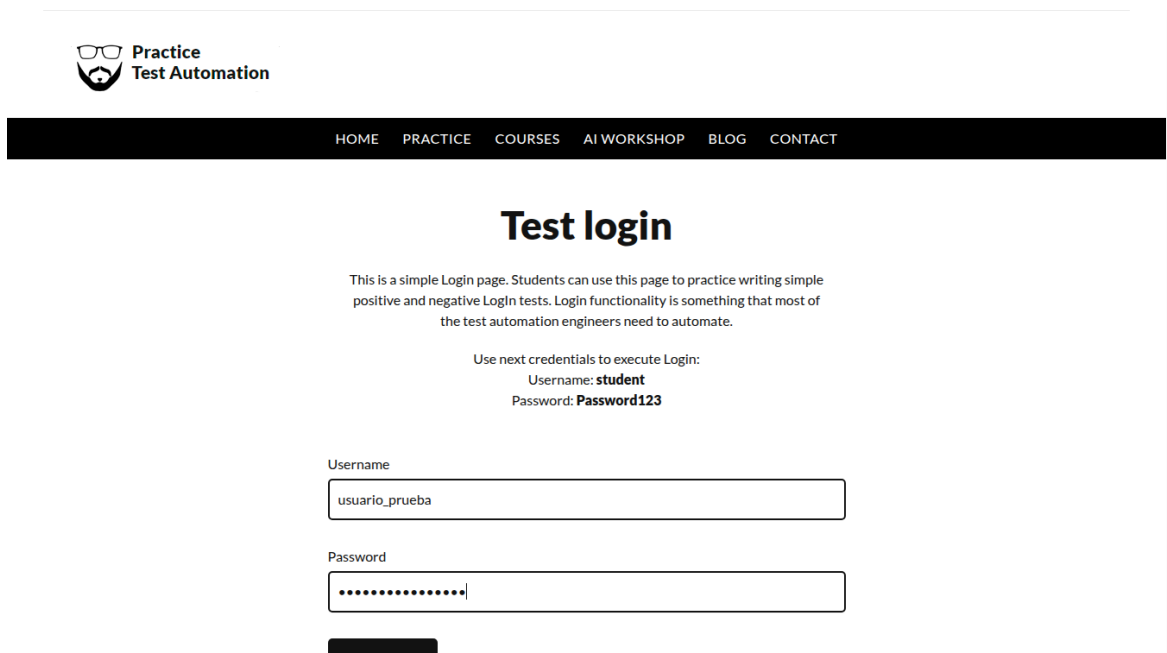


Figura 5. Formulario con credenciales invalidas antes de enviar.

The screenshot shows a web application with a black navigation bar at the top containing links: HOME, PRACTICE, COURSES, AI WORKSHOP, BLOG, and CONTACT. The main heading is 'Test login'. Below it, a paragraph explains the page's purpose for practicing login tests. It then provides example credentials: Username: **student** and Password: **Password123**. The login form consists of two input fields labeled 'Username' and 'Password', followed by a 'Login' button. A red error message is displayed below the button, indicating an invalid login attempt.

Figura 6. Mensaje de error mostrado para login invalido.

5. Resultado de la prueba

La ejecución con pytest finalizó correctamente con dos escenarios aprobados. También se generó el archivo resultado_login_valido.html como evidencia adicional del estado final del login exitoso.

Comando	pytest -v
Resultado	2 passed
Framework	pytest 8.3.4 + selenium 4.27.1
Archivo HTML	evidencias/resultado_login_valido.html

Anexo - Código fuente principal

Archivo: tests/test_practice_login.py

```
1. from pathlib import Path
2.
3. import pytest
4. from selenium import webdriver
5. from selenium.webdriver.chrome.options import Options
6. from selenium.webdriver.common.by import By
7. from selenium.webdriver.support import expected_conditions as condition
8. from selenium.webdriver.support.ui import WebDriverWait
9.
10.
11. GRID_ENDPOINT = "http://localhost:4444/wd/hub"
12. LOGIN_PAGE = "https://practicetestautomation.com/practice-test-login/"
13. OUTPUT_DIR = Path(__file__).resolve().parents[1] / "evidencias"
14.
15.
16. @pytest.fixture(scope="session", autouse=True)
17. def prepare_evidence_folder():
18.     OUTPUT_DIR.mkdir(exist_ok=True)
19.
20.
21. @pytest.fixture()
22. def browser():
23.     settings = Options()
24.     settings.add_argument("--window-size=1366,900")
25.
26.     session = webdriver.Remote(
27.         command_executor=GRID_ENDPOINT,
28.         options=settings,
29.     )
30.
31.     yield session
32.
33.     session.quit()
34.
35.
36. def fill_login_form(browser, username, password):
37.     wait = WebDriverWait(browser, 15)
38.     browser.get(LOGIN_PAGE)
39.
40.     username_box = wait.until(
41.         condition.visibility_of_element_located((By.ID, "username"))
42.     )
43.     password_box = browser.find_element(By.ID, "password")
44.
45.     username_box.clear()
46.     username_box.send_keys(username)
47.
48.     password_box.clear()
49.     password_box.send_keys(password)
50.
51.     return browser.find_element(By.ID, "submit")
52.
53.
54. def test_usuario_valido_accede_al_area_privada(browser):
55.     wait = WebDriverWait(browser, 15)
56.
57.     submit_button = fill_login_form(
58.         browser,
59.         username="student",
```

```

60.         password="Password123",
61.     )
62.
63.     browser.save_screenshot(str(OUTPUT_DIR / "01_formulario_login_valido.png"))
64.     submit_button.click()
65.
66.     success_message = wait.until(
67.         condition.visibility_of_element_located((By.TAG_NAME, "h1"))
68.     )
69.     logout_button = wait.until(
70.         condition.element_to_be_clickable((By.LINK_TEXT, "Log out"))
71.     )
72.
73.     assert "logged in successfully" in success_message.text.lower()
74.     assert "logged-in-successfully" in browser.current_url
75.     assert logout_button.is_displayed()
76.
77.     browser.save_screenshot(str(OUTPUT_DIR / "02_login_valido_confirmado.png"))
78.     (OUTPUT_DIR / "resultado_login_valido.html").write_text(
79.         browser.page_source,
80.         encoding="utf-8",
81.     )
82.
83.
84. def test_credenciales_invalidas_muestran_alerta(browser):
85.     wait = WebDriverWait(browser, 15)
86.
87.     submit_button = fill_login_form(
88.         browser,
89.         username="usuario_prueba",
90.         password="clave_equivocada",
91.     )
92.
93.     browser.save_screenshot(str(OUTPUT_DIR / "03_formulario_login_invalido.png"))
94.     submit_button.click()
95.
96.     alert_text = wait.until(
97.         condition.visibility_of_element_located((By.ID, "error"))
98.     )
99.
100.    assert "username is invalid" in alert_text.text.lower()
101.    assert "practice-test-login" in browser.current_url
102.
103.    browser.save_screenshot(str(OUTPUT_DIR / "04_login_invalido_alerta.png"))
104.

```